

박안석 진선우 이재현 손재강

KUGGLE 14th 4조

데이터 기반 뉴욕 Airbnb 숙소 추천 시스템 구축



01
주제 및 분석 개요

03
머신러닝 학습 및 가중치 도출

05
결론

02
데이터 전처리

04
분석 마무리

뉴욕 에어비앤비 숙소의 인기 요인 분석 및 가격 대비 가치 기반 추천 모델 설계

에어비앤비 숙소 선택 시 사용자는 가격, 위치, 평점 등
다양한 요소를 고려함

- ① 숙소의 인기에 영향을 주는 핵심 요인 도출 (리뷰 수, 평점, 위치, 가격 등)
- ② 가격 대비 높은 인기를 가지는 숙소의 특성 분석
- ③ 사용자 선호를 반영한 맞춤형 숙소 추천 기준 제시

Column — New York City Airbnb Open Data

- | | |
|-----------------------|---------------------------------|
| - id | - room_type |
| - name | - price |
| - host_id | - minimum_nights |
| - host_name | - number_of_reviews |
| - neighbourhood_group | - last_review |
| -neighbourhood | -reviews_per_month |
| - latitude | - calculated_host_listing_count |
| - longitude | - availability_365 |

1. 데이터 로드 — Kaggle에서 데이터셋 불러오기

```
1 import os
2 import pandas as pd
3 import re
4 import numpy as np
5 from sklearn.preprocessing import MinMaxScaler
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from sklearn.ensemble import RandomForestRegressor
9 from sklearn.model_selection import train_test_split
10 from sklearn.metrics import mean_squared_error, r2_score
11 import xgboost as xgb
12 import folium
13 # 1. Kaggle 계정 설정
14 os.environ["KAGGLE_USERNAME"] = 'AnseokPark'
15 os.environ["KAGGLE_KEY"] = "KGAT_ae85730517cf30c37b32a80ef359010b"
16
17 # 2. 뉴욕 에어비앤비 데이터셋 다운로드 경로 설정
18 # 데이터셋 주소: https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data
19 !kaggle datasets download -d dgomonov/new-york-city-airbnb-open-data -p /content/data
20
21 # 3. 압축 해제
22 !unzip -q /content/data/new-york-city-airbnb-open-data.zip -d /content/data
23
24 # 4. 파일 목록 확인 (실제 파일명이 무엇인지 확인하기 위함)
25 !ls -al /content/data
26
27 # 5. 데이터 읽기
28 df = pd.read_csv("/content/data/AB_NYC_2019.csv")
29
30 # 6. 데이터 읽기 여부 확인
31 print("데이터 로드 완료!")
```

Dataset URL: <https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data>
License(s): CC0-1.0
Downloading new-york-city-airbnb-open-data.zip to /content/data
100% 2.44M/2.44M [00:00<00:00, 139MB/s]

total 9616
drwxr-xr-x 2 root root 4096 Mar 24 11:54 .
drwxr-xr-x 1 root root 4096 Mar 24 11:54 ..
-rw-r--r-- 1 root root 7077973 Oct 27 2019 AB_NYC_2019.csv
-rw-r--r-- 1 root root 2562692 Oct 27 2019 new-york-city-airbnb-open-data.zip
-rw-r--r-- 1 root root 192340 Oct 27 2019 New_York_City_.png

데이터 로드 완료!

```
1 print(df)
2 #제대로 파일을 불러왔는지 확인
```

	id		name	host_id	#
0	2539	Clean & quiet apt home by the park		2787	
1	2595	Skylit Midtown Castle		2845	
2	3647	THE VILLAGE OF HARLEM...NEW YORK !		4632	
3	3831	Cozy Entire Floor of Brownstone		4869	
4	5022	Entire Apt: Spacious Studio/Loft by central park		7192	
...	...	...	...	...	...
48890	36484665	Charming one bedroom - newly renovated rowhouse		8232441	
48891	36485057	Affordable room in Bushwick/East Williamsburg		6570630	
48892	36485431	Sunny Studio at Historical Neighborhood		23492952	
48893	36485609	43rd St. Time Square-cozy single bed		30885759	
48894	36487245	Trendy duplex in the very heart of Hell's Kitchen		68119814	

	host_name	neighbourhood_group	neighbourhood	latitude	#
0	John	Brooklyn	Kensington	40.64749	
1	Jennifer	Manhattan	Midtown	40.75362	
2	Elisabeth	Manhattan	Harlem	40.80902	
3	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	
4	Laura	Manhattan	East Harlem	40.79851	
...	...	...	...	...	...
48890	Sabrina	Brooklyn	Bedford-Stuyvesant	40.67853	
48891	Marisol	Brooklyn	Bushwick	40.70184	
48892	Ilgar & Aysel	Manhattan	Harlem	40.81475	
48893	Taz	Manhattan	Hell's Kitchen	40.75751	
48894	Christophe	Manhattan	Hell's Kitchen	40.76404	

	longitude	room_type	price	minimum_nights	number_of_reviews	#
0	-73.97237	Private room	149	1	9	
1	-73.98377	Entire home/apt	225	1	45	
2	-73.94190	Private room	150	3	0	
3	-73.95976	Entire home/apt	89	1	270	
4	-73.94399	Entire home/apt	80	10	9	
...	...	...	...	...	...	...
48890	-73.94995	Private room	70	2	0	
48891	-73.93317	Private room	40	4	0	
48892	-73.94867	Entire home/apt	115	10	0	
48893	-73.99112	Shared room	55	1	0	
48894	-73.98933	Private room	90	7	0	

	last_review	reviews_per_month	calculated_host_listings_count	#
0	2018-10-19	0.21	6	
1	2019-05-21	0.38	2	
2	NaN	NaN	1	
3	2019-07-05	4.64	1	
4	2018-11-19	0.10	1	
...	...	...	...	...
48890	NaN	NaN	2	
48891	NaN	NaN	2	
48892	NaN	NaN	1	
48893	NaN	NaN	6	
48894	NaN	NaN	1	

	availability_365
0	365
1	355
2	365
3	194
4	0

2. 데이터 전처리 Data Preprocessing

```
1 df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
2 df = df[df['reviews_per_month'] > 0].copy()
3 #review_per_month의 처리가 되었는지 확인
4 df.describe()
```

	id	host_id	latitude	longitude	price	minimum_nights	number_of_reviews	reviews_per_month	calculated_host_listings_count	availability_365
count	3.884300e+04	3.884300e+04	38843.000000	38843.000000	38843.000000	38843.000000	38843.000000	38843.000000	38843.000000	38843.000000
mean	1.809646e+07	6.423915e+07	40.728134	-73.951148	142.317947	5.868059	29.297557	1.373221	5.164457	114.882888
std	1.069370e+07	7.588847e+07	0.054990	0.046695	196.945624	17.384784	48.186374	1.680442	26.295665	129.543636
min	2.539000e+03	2.438000e+03	40.506410	-74.244420	0.000000	1.000000	1.000000	0.010000	1.000000	0.000000
25%	8.720027e+06	7.033824e+06	40.688640	-73.982470	69.000000	1.000000	3.000000	0.190000	1.000000	0.000000
50%	1.887146e+07	2.837193e+07	40.721710	-73.954800	101.000000	2.000000	9.000000	0.720000	1.000000	55.000000
75%	2.755482e+07	1.018465e+08	40.762990	-73.935020	170.000000	4.000000	33.000000	2.020000	2.000000	229.000000
max	3.645581e+07	2.738417e+08	40.913060	-73.712990	10000.000000	1250.000000	629.000000	58.500000	327.000000	365.000000

- 데이터셋 중 review_per_month의 결측치를 0으로 대체하여 '실제로 활동중인 숙소들' 을 대상으로 전처리 진행
- '뉴욕의 모든 에어비앤비' 에서 '뉴욕에서 실제로 사람들이 이용하는 에어비앤비' 로 타겟을 변경
 - 모델이 이해할 수 있도록 범주형 데이터를 수치화함

3. 새로운 파생변수 생성 Feature Engineering for Popularity prediction

```
df['name'] = df['name'].fillna('').astype(str)
df['name_clean'] = df['name'].apply(lambda x: re.sub(r'^a-zA-Z#s]', '', x).lower())

# 1. 마케팅 키워드 설정(임의로 설정함, 추후 변경가능)
marketing_keywords = ['cozy', 'modern', 'subway', 'spacious', 'quiet', 'clean', 'luxury', 'central', 'private', 'bright']
df['keyword_count'] = df['name_clean'].apply(lambda text: sum(1 for word in marketing_keywords if word in text))

# 2. 지역명 포함 여부
df['contains_borough'] = df.apply(lambda row: 1 if row['neighbourhood_group'].lower() in row['name'].lower() else 0, axis=1)

# 3. 평균 가격 대비 비율 (로그 변환 적용)
neighborhood_avg = df.groupby('neighbourhood_group')['price'].transform('mean')
df['price_ratio_raw'] = df['price'] / neighborhood_avg
df['price_ratio_log'] = np.log1p(df['price_ratio_raw'])
```

1. 마케팅 키워드 개수 (keyword_count) : 'Cozy', 'Modern', 'Near Subway' 같이 긍정적인 단어가 포함이 되어있는 집의 경우, 영어권 게스트들에게 어필이 될 수 있다고 판단하여 변수로 생성
2. 지역명 중복 여부 (contains_borough) : 숙소 이름에 지역명이 넣어져 있는 경우, 검색 노출을 노린 '마케팅'으로 생각하여 변수로 생성
3. 지역 평균 대비 가격 비율 (price_ratio_neighborhood) : 해당 숙소가 속한 지역의 평균 가격 대비 얼마나 저렴하거나 비싼지를 계산해 단순히 가격순이 아니라 특정 지역에 있는 숙소들 대비 가성비가 얼마나 좋은지를 판단하는 지표라고 생각해 변수로 생성

* 평균 가격 대비 비율에만 로그 변환을 적용한 이유는 전처리 과정에서 극단치로 인해 데이터의 스케일링이 적용되지 않아 로그 변환을 적용함

4. 클리핑 및 원-핫 인코딩 진행 → 범주형 변수는 모델이 이해할 수 없기 때문에 수치형태로 변환

```
# 분석 타겟인 reviews_per_month를 포함해 클리핑을 진행
df['price'] = np.clip(df['price'], 0, df['price'].quantile(0.99))
df['minimum_nights'] = np.clip(df['minimum_nights'], 0, df['minimum_nights'].quantile(0.99))
df['reviews_per_month'] = np.clip(df['reviews_per_month'], 0, df['reviews_per_month'].quantile(0.99))
```

↑ 극단치 값을 없애는 클리핑을 진행함

```
# 타겟 변수(reviews_per_month)는 직관성을 위해 스케일링에서 제외합니다.
scaling_cols = ['price', 'minimum_nights', 'availability_365', 'keyword_count', 'price_ratio_log']

scaler = MinMaxScaler()
df[scaling_cols] = scaler.fit_transform(df[scaling_cols])

# 머신러닝에 넣을 최종 컬럼만 선택
final_cols = scaling_cols + ['contains_borough', 'neighbourhood_group', 'room_type', 'reviews_per_month']

# 범주형 데이터 원-핫 인코딩 (df_final 완성)
df_final = pd.get_dummies(df[final_cols], columns=['neighbourhood_group', 'room_type'], drop_first=True)

1 print("최종 데이터프레임(df_final) 크기 :", df_final.shape)
2 display(df_final.head(3))
```

↑ 데이터 스케일링 및 원-핫 인코딩을 진행해 최종 데이터프레임을 생성

4. 클리핑 및 원-핫 인코딩 진행 Clipping & One-Hot Encoding

최종 데이터프레임(df_final) 크기: (38843, 13)

	price	minimum_nights	availability_365	keyword_count	price_ratio_log	contains_borough	reviews_per_month	neighbourhood_group_Brooklyn
0	0.222004	0.0	1.000000	0.50	0.171874	0	0.21	True
1	0.335240	0.0	0.972603	0.00	0.174059	0	0.38	False
3	0.132606	0.0	0.531507	0.25	0.118023	0	4.64	True

neighbourhood_group_Manhattan	neighbourhood_group_Queens	neighbourhood_group_Staten Island	room_type_Private room	room_type_Shared room
False	False	False	True	False
True	False	False	False	False
False	False	False	False	False

↑ 최종 데이터프레임 (ex. room_type = "private room" → [0, 1, 0])

모델이 안정적으로 학습할 수 있도록 데이터 분포와 구조를 정제하였으며
이 과정을 통해 머신러닝 학습의 성능을 향상시킬 수 있습니다.

5. 데이터 시각화 코드 Exploratory Data Analysis (EDA)

```

1  fig = plt.figure(figsize=(18, 12))
2  sns.set_theme(style="whitegrid")
3
4
5  # 1. 연속형 데이터 분포 (히스토그램)
6
7  # 앞서 전처리 과정을 거친 컬럼들로 진행함.
8  numeric_cols = ['price', 'minimum_nights', 'availability_365', 'keyword_count', 'price_ratio_log', 'reviews_per_month']
9
10 for i, col in enumerate(numeric_cols):
11     ax = fig.add_subplot(2, 4, i+1) # 2행 4열 배치
12     # 리뷰 수는 색상을 다르게 하여 타겟 변수임을 강조
13     color = 'coral' if col == 'reviews_per_month' else 'skyblue'
14     sns.histplot(df[col], bins=30, kde=True, color=color, ax=ax)
15
16     title_prefix = "[Target]" if col == 'reviews_per_month' else "[Scaled]"
17     ax.set_title(f'{title_prefix} {col}', fontsize=11, fontweight='bold')
18     ax.set_xlabel('')
19     ax.set_ylabel('')
20
21
22 # 2. 범주형 데이터 분포 (파이 차트)
23
24 # 파이 차트는 원본 텍스트가 살아있는 df를 활용합니다.
25
26 # 지역별 비율 (neighbourhood_group)
27 ax7 = fig.add_subplot(2, 4, 7)
28 region_counts = df['neighbourhood_group'].value_counts()
29 ax7.pie(region_counts, labels=region_counts.index, autopct='%1.1f%%', startangle=90, colors=sns.color_palette('pastel'))
30 ax7.set_title('[Category] Neighbourhood Group', fontsize=11, fontweight='bold')
31
32 # 방 타입별 비율 (room_type)
33 ax8 = fig.add_subplot(2, 4, 8)
34 room_counts = df['room_type'].value_counts()
35 ax8.pie(room_counts, labels=room_counts.index, autopct='%1.1f%%', startangle=90, colors=sns.color_palette('Set2'))
36 ax8.set_title('[Category] Room Type', fontsize=11, fontweight='bold')
37
38 plt.tight_layout()
39 plt.show()

```

연속형 데이터 분포 (히스토그램)

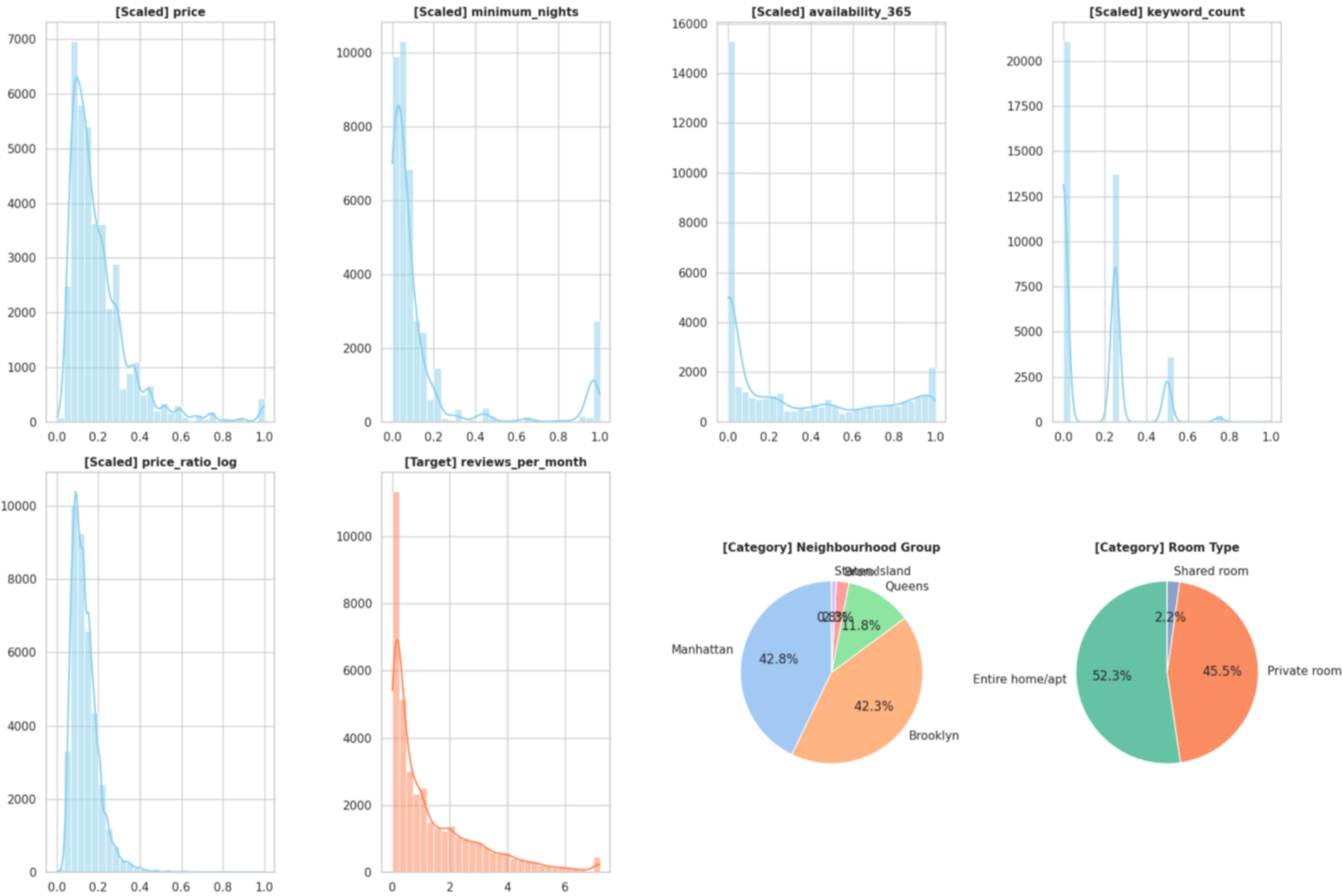
- 2행 4열 배치
- 리뷰 수는 코랄색, 다른 변수는 하늘색으로 색상을 다르게 하여 타겟 변수 강조

범주형 데이터 분포 (파이 차트)

- 지역별 비율 (neighbourhood_group)
- 방 타입별 비율 (room_type)

↑ df_final에 있는 데이터들을 시각적으로 보기 위해 변수 시각화 코드를 작성

5. 데이터 시각화



1. RandomForest 모델 학습 및 성능 평가 진행

```
# 1. 스케일링 대상 재지정
scaling_cols = [
    'price', 'minimum_nights', 'availability_365', 'keyword_count', 'price_ratio_log',
    'latitude', 'longitude', 'calculated_host_listings_count' # 추가된 핵심 공간 & 호스트 정보
]

# (df는 0값 제거 및 이상치 처리가 완료된 상태라고 가정합니다)
scaler = MinMaxScaler()
df[scaling_cols] = scaler.fit_transform(df[scaling_cols])

# 2. 최종 컬럼 세팅 및 원-핫 인코딩
final_cols = scaling_cols + ['contains_borough', 'neighbourhood_group', 'room_type', 'reviews_per_month']
df_final = pd.get_dummies(df[final_cols], columns=['neighbourhood_group', 'room_type'], drop_first=True)

print("\nRandom Forest 모델 최종 평가 결과")

# 3. 정답지(Target)에 로그 변환 적용
X = df_final.drop('reviews_per_month', axis=1)
y_log = np.log1p(df_final['reviews_per_month']) # 로그 씌우기

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y_log, test_size=0.2, random_state=42)

# 4. 하이퍼파라미터가 튜닝된 덜렁대지 않는 Random Forest 모델!
# max_depth와 min_samples_split을 주어 과적합을 방지.
rf_model_tuned = RandomForestRegressor(
    n_estimators=200,      # 나무를 200개로 늘려 안정성 확보
    max_depth=15,         # 나무 깊이 제한
    min_samples_split=10,  # 너무 작은 가지치기 방지
    random_state=42,
    n_jobs=-1
)
rf_model_tuned.fit(X_train, y_train)
```

'latitude', 'longitude', 'calculated_host_listings_count'
추가하여 스케일링 대상을 재지정

범주형 데이터를 원-핫 인코딩을 통해 0과 1로 변환
* drop_first=True는 다중공선성을 방지하기 위해 첫 번째 카테고리를 제외함

월별 리뷰 수가 한쪽으로 치우친 분포를 띠고 있을 확률 높음
→ np.log1p를 사용해 정규분포에 가깝게 함
(0값 오류를 막기 위해 일반 log가 아닌 log1p 사용)

train data와 test data를 8:2로 나눔
랜덤 포레스트를 사용 → 기본값을 쓰지 않고 파라미터를
제어하여 과적합 현상을 방지함

1. RandomForest 모델 학습 및 성능 평가 진행

```
# 5. 성능 확인 (로그를 다시 원래 숫자로 되돌려서 계산)
y_pred_log = rf_model_tuned.predict(X_test)
y_pred_real = np.expml(y_pred_log) # expml으로 로그를 풀어줌
y_test_real = np.expml(y_test)

rmse = np.sqrt(mean_squared_error(y_test_real, y_pred_real))
r2 = r2_score(y_test_real, y_pred_real)

print(f"오차(RMSE): {rmse:.4f}")
print(f"설명력(R2 Score): {r2:.4f}\n")

# 6. 새로운 가중치(Feature Importance) 확인
importances = rf_model_tuned.feature_importances_
feature_importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, color='cornflowerblue')
plt.title('Tuned Feature Importance (Weights)')
plt.tight_layout()
plt.show()
```

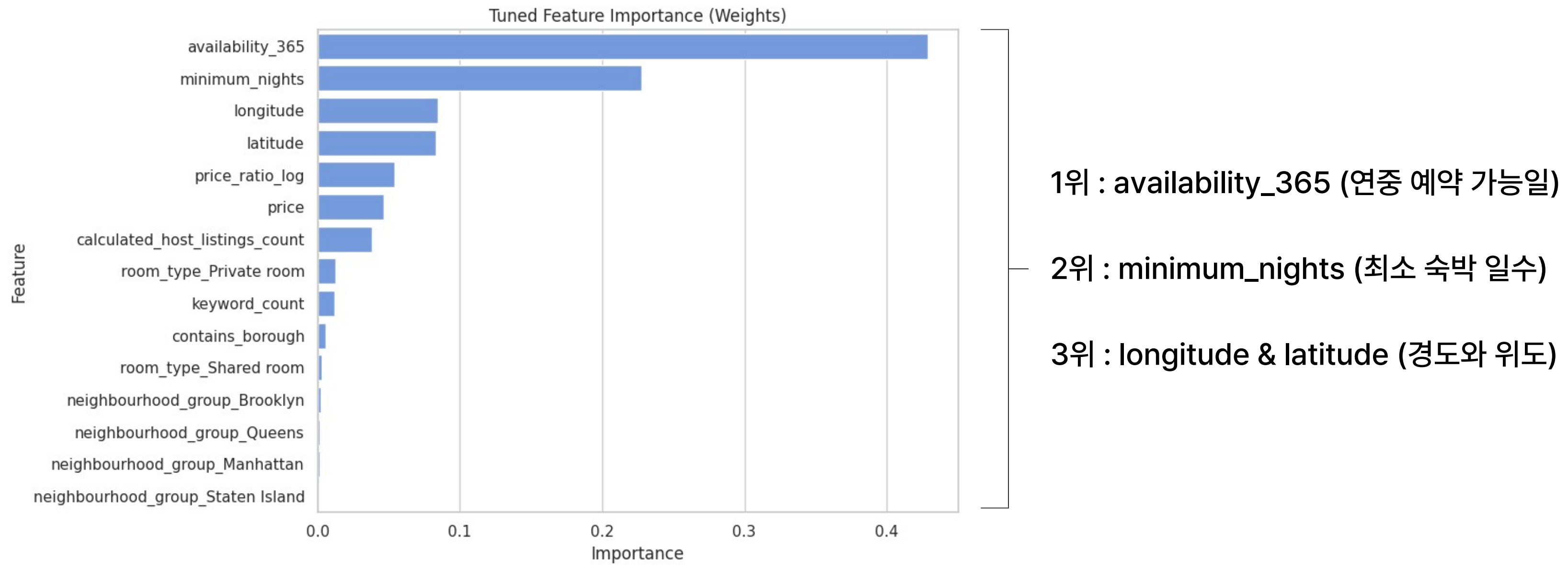
평가지표의 직관성을 위해 지수변환(expml)으로 로그를 풀어 원래 수치 단위로 복원한 후, RMSE와 R2를 계산

랜덤 포레스트 모델이 타겟(리뷰 수)을 예측할 때 어떤 변수를 가장 중요하게 생각했는지 가중치를 뽑아내어 바 차트로 생성

Random Forest 모델 최종 평가 결과
오차(RMSE): 1.2334
설명력(R2 Score): 0.3731

→ "RandomForest 모델은 기본적인 숙소 환경 데이터를 기반으로 리뷰 수의 흐름을 37%가량 설명하고 있음.
비록 오차가 1.2회 정도 발생하지만, 변수 중요도를 참고하면 개선이 필요한 요소의 전략적 인사이트를 얻기에 충분함.

1. RandomForest 모델 학습 및 성능 평가 진행



2. XGBoost 모델 학습 및 성능 평가 진행

```
# 1. XGBoost 모델 정의 (하이퍼파라미터 세팅)
# Random Forest와 달리 오답을 고쳐나가며 섬세하게 학습하도록 설정합니다.
xgb_model = xgb.XGBRegressor(
    n_estimators=300,      # 나무의 개수 (오답을 고칠 기회 300번)
    learning_rate=0.05,    # 학습률 (0.05의 속도로 천천히, 꼼꼼하게 학습)
    max_depth=8,           # 나무의 최대 깊이 (너무 깊으면 과적합되므로 8로 제한)
    subsample=0.8,         # 매번 전체 데이터의 80%만 뽑아서 학습 (과적합 방지)
    colsample_bytree=0.8,   # 매번 변수의 80%만 뽑아서 학습 (다양한 관점 확보)
    random_state=42,
    n_jobs=-1              # 컴퓨터의 모든 코어를 사용해 속도 향상
)

# 2. 모델 학습
print("XGBoost 모델 학습 시작")
xgb_model.fit(X_train, y_train)

# 3. 예측 및 성능 평가 (로그 변환 복구)
# 모델은 로그가 씌워진 값을 예측하므로, 다시 원래 숫자로 돌려줍니다.
y_pred_xgb_log = xgb_model.predict(X_test)
y_pred_xgb_real = np.expm1(y_pred_xgb_log) # expm1으로 로그 풀기

# 성능 계산
rmse_xgb = np.sqrt(mean_squared_error(y_test_real, y_pred_xgb_real))
r2_xgb = r2_score(y_test_real, y_pred_xgb_real)

print("#XGBoost 모델 최종 평가 결과")
print(f"오차(RMSE): {rmse_xgb:.4f}")
print(f"설명력(R2 Score): {r2_xgb:.4f}")

# 4. XGBoost가 뽑아낸 변수 중요도(가중치) 시각화
xgb_importances = xgb_model.feature_importances_
xgb_importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': xgb_importances
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(10, 6))
# Random Forest와 비교하기 쉽게 색상을 다르게 지정.
sns.barplot(x='Importance', y='Feature', data=xgb_importance_df, palette='magma')
plt.title('XGBoost Feature Importance (Weights)')
plt.tight_layout()
plt.show()
```

이전 모델보다 더 정교하게 예측하기 위해 XGBoost 모델을 사용
오답을 집중적으로 다시 공부해서 오차를 줄여나가는 방식 도입

이전과 마찬가지로 로그를 씌워 학습했으므로 결과물도 기존으로
돌려주는 작업을 진행하고 XGBoost의 성능 계산

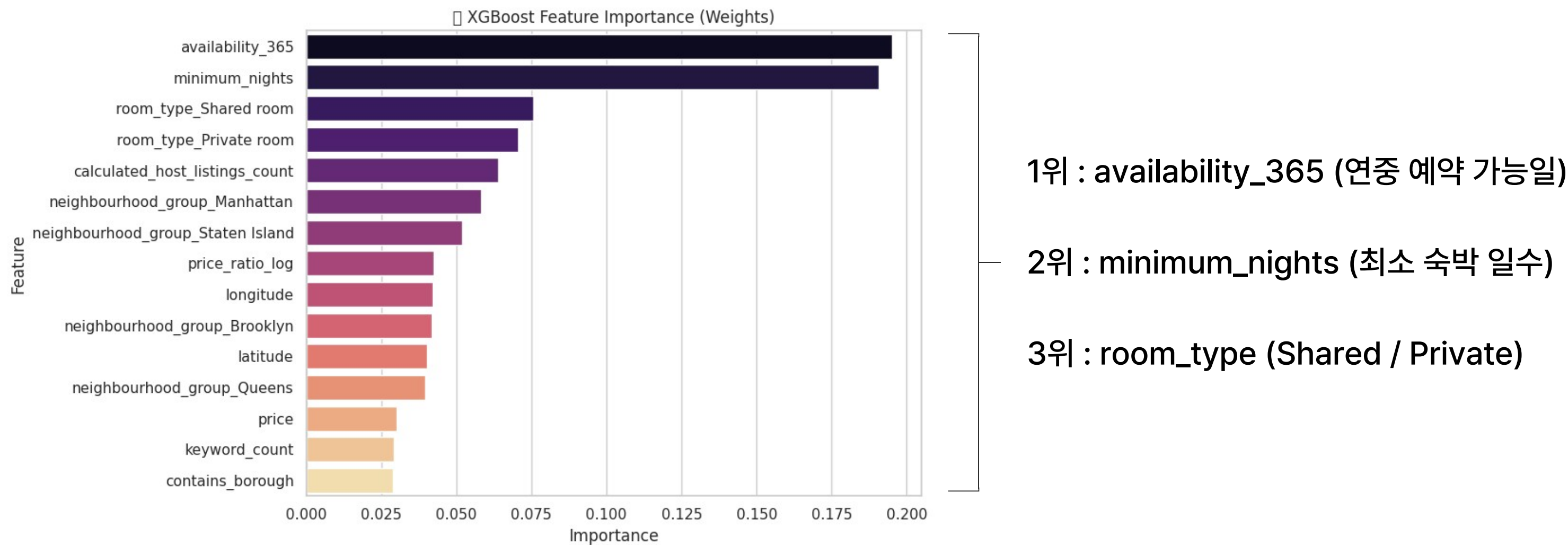
XGBoost가 뽑아낸 변수의 가중치를 시각화하는 작업을 진행
(RandomForest와 비교하기 위해 색상에 차별점을 둬)

2. XGBoost 모델 학습 및 성능 평가 진행

```
XGBoost 모델 학습 시작

XGBoost 모델 최종 평가 결과
오차(RMSE): 1.2108
설명력(R2 Score): 0.3959
```

→ XGBoost의 RMSE(1.2108)의 경우 Random Forest(1.2334)보다 약 0.02 정도 오차가 감소
R2의 경우 Random Forest(37.31%) 대비 설명력이 약 2.3%p 상승



1. 맞춤형 추천 리스트 작성

```
print("뉴욕 에어비앤비 맞춤형 추천 시스템")

# 1. XGBoost 모델의 가중치 가져오기
xgb_importances = xgb_model.feature_importances_
xgb_weights = pd.Series(xgb_importances, index=X.columns)

# 2. 추천 점수 산출 함수 정의
def get_xgb_recommendation(theme_name, target_feature=None, weight_multiplier=3.0, top_n=10):
    weights = xgb_weights.copy()
    X_calc = X.copy()

    # [핵심 로직] 특정 테마에 가중치 부여 및 방향성 조정
    if target_feature:
        if target_feature == 'price_ratio_log':
            # 가성비: 가격 비율이 '낮을수록' 좋은 숙소이므로 (1 - 값)으로 뒤집어서 점수화
            X_calc['price_ratio_log'] = 1.0 - X_calc['price_ratio_log']
            weights[target_feature] *= weight_multiplier
        elif target_feature in weights.index:
            # 다른 마케팅 지표 등은 높을수록 좋으므로 가중치만 증폭
            weights[target_feature] *= weight_multiplier

    # 3. 점수 계산 (행렬 곱)
    raw_scores = X_calc.dot(weights)

    # 4. 0 ~ 100점 만점으로 스케일링
    min_score = raw_scores.min()
    max_score = raw_scores.max()
    final_scores = ((raw_scores - min_score) / (max_score - min_score)) * 100
```

모델에서 도출된 변수 중요도를 기반으로 숙소의 '추천점수'를 설계

- 가성비 = '1 - 가격' 으로 점수화 (!)
- 가중치를 증폭시켜 나타내도록 함

각 숙소가 가진 특징값들과 모델의 가중치를 곱해서 합함
결과값을 0~100점 사이의 점수로 만들기 위해 스케일링 진행

1. 맞춤형 추천 리스트 작성

```
# 5. 원본 데이터프레임과 결합 (지도 시각화를 위해 위도, 경도 필수 포함!)
display_cols = ['name', 'neighbourhood_group', 'room_type', 'price', 'latitude', 'longitude']
result_df = df.loc[df_final.index, display_cols].copy()
result_df['Theme'] = theme_name
result_df['Score'] = final_scores.round(1)

# 6. 상위 N개 추출
return result_df.sort_values(by='Score', ascending=False).head(top_n)

# (1) 종합 밸런스 점수 TOP 10 (가중치 조작 없음)
top10_balanced = get_xgb_recommendation(theme_name='종합 베스트', top_n=10)
print("\n[ 종합 베스트 TOP 10 ]")
display(top10_balanced[['name', 'neighbourhood_group', 'price', 'Score']])

# (2) 가성비 중시형 TOP 10 (price_ratio_log 가중치 3배)
top10_value = get_xgb_recommendation(theme_name='가성비 끝판왕', target_feature='price_ratio_log', weight_multiplier=3.0, top_n=10)
print("\n[ 가성비 TOP 10 ]")
display(top10_value[['name', 'neighbourhood_group', 'price', 'Score']])

# (3) 마케팅 우수형 TOP 10 (keyword_count 가중치 3배)
top10_marketing = get_xgb_recommendation(theme_name='마케팅 우수형', target_feature='keyword_count', weight_multiplier=3.0, top_n=10)
print("\n[ 마케팅 우수형 TOP 10 ]")
display(top10_marketing[['name', 'neighbourhood_group', 'price', 'Score']])

# 다음 단계(지도 시각화)를 위해 3개의 데이터프레임을 하나로 합쳐줍니다.
final_top30_map_data = pd.concat([top10_balanced, top10_value, top10_marketing])
```

종합 베스트 : 모든 변수를 균형 있게 반영한 기본 추천 리스트
가성비 : 가격 대비 효율을 중시하는 사용자에게 적합한 추천 리스트
마케팅 : 숙소 정보량 및 표현력이 높은 숙소를 강조한 추천 리스트

가중치를 증폭시킨 경우

↑ 이전 결과물을 원본 데이터프레임과 결합하여 고객 타입별 상위 N개 추출

1. 맞춤형 추천 리스트 작성

뉴욕 에어비앤비 맞춤형 추천 시스템

[종합 베스트 TOP 10]

	name	neighbourhood_group	price	Score
46796	Modern designed room in Manhattan! near B/C tr...	Manhattan	0.132606	100.0
13788	Private Room in Williamsburg!	Brooklyn	1.000000	99.195019
15890	Private Room in Williamsburg Duplex	Brooklyn	1.000000	99.184006
1358	1 Bed Manhattan Apt. Minimum 7 DAYS	Manhattan	0.089397	99.039591
6349	Location Top Manhattan	Manhattan	0.148996	98.907307
2396	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	Manhattan	0.065558	98.55918
40909	Private room with a great location in Manhattan	Manhattan	0.087908	96.553251
655	At a very nice area in the WestSide	Manhattan	0.186245	96.299695
40197	Very Clean and Very Large	Manhattan	0.148996	95.95128
42409	SPACIOUS SUNNY ROOM NEAR SUBWAY AND HUDSON rIVER	Manhattan	0.077478	95.942738

[가성비 TOP 10]

	name	neighbourhood_group	price	Score
46796	Modern designed room in Manhattan! near B/C tr...	Manhattan	0.132606	100.0
1358	1 Bed Manhattan Apt. Minimum 7 DAYS	Manhattan	0.089397	99.828177
2396	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	Manhattan	0.065558	99.818298
6349	Location Top Manhattan	Manhattan	0.148996	98.721471
40909	Private room with a great location in Manhattan	Manhattan	0.087908	97.527065
19772	1.5 BATHRM SUNNY SPACIOUS ROOM BY SUBWAY	Manhattan	0.074498	97.189739
42409	SPACIOUS SUNNY ROOM NEAR SUBWAY AND HUDSON rIVER	Manhattan	0.077478	97.145028
12310	Near Hudson River and Subway quiet apt	Manhattan	0.074498	96.632401
40981	Vibrant Private Bedroom in UWS 107	Manhattan	0.089397	96.372159
16157	Glamorous shared room in Manhattan	Manhattan	0.067048	96.132295

[마케팅 우수형 TOP 10]

	name	neighbourhood_group	price	Score
2396	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	Manhattan	0.065558	100.0
46796	Modern designed room in Manhattan! near B/C tr...	Manhattan	0.132606	96.237785
13788	Private Room in Williamsburg!	Brooklyn	1.000000	95.483665
15890	Private Room in Williamsburg Duplex	Brooklyn	1.000000	95.473347
42409	SPACIOUS SUNNY ROOM NEAR SUBWAY AND HUDSON rIVER	Manhattan	0.077478	94.99287
19772	1.5 BATHRM SUNNY SPACIOUS ROOM BY SUBWAY	Manhattan	0.074498	94.982373
12310	Near Hudson River and Subway quiet apt	Manhattan	0.074498	94.424672
29585	Bright-Spacious Private Room /30 min NYC	Queens	0.080458	94.350087
40863	Columbia & CentralPark Private room!	Manhattan	0.081948	93.60066
98	COZY QUIET room 4 DOOGLERS!	Manhattan	0.146016	93.472375

상위 숙소들은 대부분 Manhattan 지역에 집중되어 있음
추천 기준에 따라 상위 숙소 구성이 달라지는 것을 확인할 수 있음
→ 이는 사용자 목적에 따라 최적의 숙소가 달라질 수 있음을 의미함

2. 지역별 추천 리스트 작성

```
print("지역별(Neighbourhood Group) 종합 점수 TOP 5 숙소")

# 1. XGBoost 모델의 가중치를 활용하여 전체 숙소의 종합 점수(100점 만점) 일괄 계산
xgb_weights = pd.Series(xgb_model.feature_importances_, index=X.columns)
raw_scores = X.dot(xgb_weights)

min_s = raw_scores.min()
max_s = raw_scores.max()
final_scores = ((raw_scores - min_s) / (max_s - min_s)) * 100

# 2. 결과 출력을 위한 데이터프레임 구성
display_cols = ['name', 'neighbourhood_group', 'room_type', 'price', 'latitude', 'longitude']
result_df = df.loc[df_final.index, display_cols].copy()

# 소수점 첫째 자리까지 반올림하여 점수 저장
result_df['종합점수(Score)'] = final_scores.round(1)

# 에러 해결용 코드
# nlargest가 작동할 수 있도록 확실하게 숫자형(float)으로 강제 변환
result_df['종합점수(Score)'] = pd.to_numeric(result_df['종합점수(Score)'], errors='coerce')

# 3. 각 지역별로 그룹을 묶은 뒤, 종합점수 기준 상위 5개씩만 뽑아내기
top5_per_borough = result_df.groupby('neighbourhood_group', group_keys=False).apply(
    lambda x: x.nlargest(5, '종합점수(Score)')
)

# 4. 출력하기
boroughs = top5_per_borough['neighbourhood_group'].unique()

for borough in boroughs:
    print(f"📍 [ {borough} ] 지역 최고 인기 잠재력 TOP 5")

    # 해당 지역의 데이터만 필터링
    borough_data = top5_per_borough[top5_per_borough['neighbourhood_group'] == borough].copy()

    # Raw 가격을 직관적으로 보여주기 위해 $ 기호와 쉼표 추가 (출력용으로만 문자열로 변환)
    borough_data['실제 가격(Raw)'] = borough_data['price'].apply(lambda x: f"${x:,.0f}")

    # 출력할 컬럼만 선택하여 인덱스를 깔끔하게 재정렬 후 출력
    final_display_cols = ['name', 'room_type', '실제 가격(Raw)', '종합점수(Score)']
    display(borough_data[final_display_cols].reset_index(drop=True))

# 5. 다음 단계인 지도 시각화를 위해 이 데이터를 따로 저장해 두기.
map_data_borough_top5 = top5_per_borough.copy()
```

학습된 XGBoost 모델의 가중치를 가져와 뉴욕에 있는 모든 숙소 데이터와 곱함
이를 다시 0~100점 사이로 변환 (Min-Max Scaling)

점수가 문자열로 인식되어 정렬 오류가 나는 것을 방지하기 위해 숫자형으로 강제 변환

뉴욕의 5개 지역구별로 데이터를 묶은 뒤 점수가 가장 높은 상위 5개 숙소 선정

2. 지역별 추천 리스트 작성

	name	room_type	실제 가격 (Raw)	종합점수 (Score)
0	Cozy Private Room	Private room	\$0	82.667041
1	Blissfully Quiet Room	Private room	\$0	81.733447
2	Prolonged Traveler's Dream (a month first & more)	Private room	\$0	81.247812
3	Prolonged Traveler's Dream(a month first & more)	Private room	\$0	80.224578
4	Prolonged Traveler's Dream (a month first & more)	Private room	\$0	80.155744

 [Brooklyn] 지역 최고 인기 잠재력 TOP 5

	name	room_type	실제 가격 (Raw)	종합점수 (Score)
0	Private Room in Williamsburg!	Private room	\$1	99.195019
1	Private Room in Williamsburg Duplex	Private room	\$1	99.184006
2	Private bedroom and bathroom in Brooklyn	Private room	\$0	95.441929
3	Brooklyn Private Room 20 min to Manhattan	Private room	\$0	94.973769
4	Luxury Private Bedroom in Brooklyn 2R-1	Private room	\$0	94.228700

 [Manhattan] 지역 최고 인기 잠재력 TOP 5

	name	room_type	실제 가격 (Raw)	종합점수 (Score)
0	Modern designed room in Manhattan! near B/C tr...	Private room	\$0	100.000000
1	1 Bed Manhattan Apt. Minimum 7 DAYS	Private room	\$0	99.039591
2	Location Top Manhattan	Private room	\$0	98.907307
3	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	Private room	\$0	98.559180
4	Private room with a great location in Manhattan	Private room	\$0	96.553251

	name	room_type	실제 가격 (Raw)	종합점수 (Score)
0	Clean, Comfy Bright Small Bedroom	Private room	\$0	93.063015
1	Bright-Spacious Private Room /30 min NYC	Private room	\$0	92.528217
2	Great Bedroom in Ridgewood, Queens!	Private room	\$0	92.221811
3	One stop subway from mamhattan	Private room	\$0	91.330415
4	Renovated Private Room in Ridgewood Coliving S...	Shared room	\$0	91.182021

 [Staten Island] 지역 최고 인기 잠재력 TOP 5

	name	room_type	실제 가격 (Raw)	종합점수 (Score)
0	Share quiet, clean and spacious apt	Private room	\$0	89.262501
1	Room Available-Close to Manhattan-B	Private room	\$0	86.458059
2	Room Available Close to Manhattan-A	Private room	\$0	85.645930
3	room b	Private room	\$0	84.671430
4	1-Bedroom Newly Renovated Cozy Apartment	Entire home/apt	\$0	67.787397

3. 뉴욕 지역별 TOP 5 숙소 시각화

```
print("="*60)
print("원본 좌표 추출 및 뉴욕 지도 생성")
print("="*60)

# 1. XGBoost 모델 점수 계산
xgb_weights = pd.Series(xgb_model.feature_importances_, index=X.columns)
raw_scores = X.dot(xgb_weights)

min_s = raw_scores.min()
max_s = raw_scores.max()
final_scores = ((raw_scores - min_s) / (max_s - min_s)) * 100

# 2. 결과 출력을 위한 데이터프레임 구성 (위도, 경도는 제외)
display_cols = ['name', 'neighbourhood_group', 'room_type', 'price']
result_df = df.loc[df_final.index, display_cols].copy()

# 앞서 전처리된 위도,경도,가격 데이터 대신 원본에서 위도,경도,가격 데이터를 가져옴.
raw_data = pd.read_csv('/content/data/AB_NYC_2019.csv')
result_df['latitude'] = raw_data.loc[result_df.index, 'latitude']
result_df['longitude'] = raw_data.loc[result_df.index, 'longitude']
result_df['price_raw'] = raw_data.loc[result_df.index, 'price']
result_df['종합점수(Score)'] = pd.to_numeric(final_scores.round(1), errors='coerce')

# 점수를 숫자형으로 안전하게 저장
result_df['종합점수(Score)'] = pd.to_numeric(final_scores.round(1), errors='coerce')

# 3. 각 지역별 상위 5개 추출
top5_per_borough = result_df.groupby('neighbourhood_group', group_keys=False).apply(
    lambda x: x.nlargest(5, '종합점수(Score)')
)
```

정확한 지도 시각화를 위해 모델 학습용 데이터가 아닌 실제 뉴욕의 물리적 좌표 정보를 원본 파일에서 재매칭하여 복수

위 계산한 final_scores에서 각 지역별 상위 5개 숙소 추출

3. 뉴욕 지역별 TOP 5 숙소 시각화

```
# 4. 뉴욕 지도 생성 및 마커 추가

# 뉴욕 중심부 좌표
nyc_map = folium.Map(location=[40.7306, -73.9352], zoom_start=11, tiles='cartodbpositron')

borough_colors = {
    'Manhattan': 'red', 'Brooklyn': 'blue', 'Queens': 'green',
    'Bronx': 'purple', 'Staten Island': 'orange'
}

for idx, row in top5_per_borough.iterrows():
    borough = row['neighbourhood_group']

    popup_html = (
        f"<div style='width: 250px; font-family: Arial, sans-serif;'>"
        f"<h4 style='margin-bottom: 5px; color: #333;'><b>{row['name']}</b></h4>"
        f"<hr style='margin: 5px 0;'>"
        f"<p style='margin: 2px 0;'> <b>지역:</b> {borough}</p>"
        f"<p style='margin: 2px 0;'> <b>타입:</b> {row['room_type']}</p>"
        f"<p style='margin: 2px 0;'> <b>가격:</b> <span style='color: green; font-weight: bold;'>${row['price_raw']:, .0f}</span> / 박</p>"
        f"<p style='margin: 2px 0;'> <b>종합점수:</b> <span style='color: red; font-weight: bold;'>{row['종합점수(Score)']}점</span></p>"
        f"</div>"
    )

    folium.Marker(
        location=[row['latitude'], row['longitude']],
        popup=folium.Popup(popup_html, max_width=300),
        tooltip=f"클릭! ({borough} TOP 5)",
        icon=folium.Icon(color=borough_colors.get(borough, 'gray'), icon='star')
    ).add_to(nyc_map)

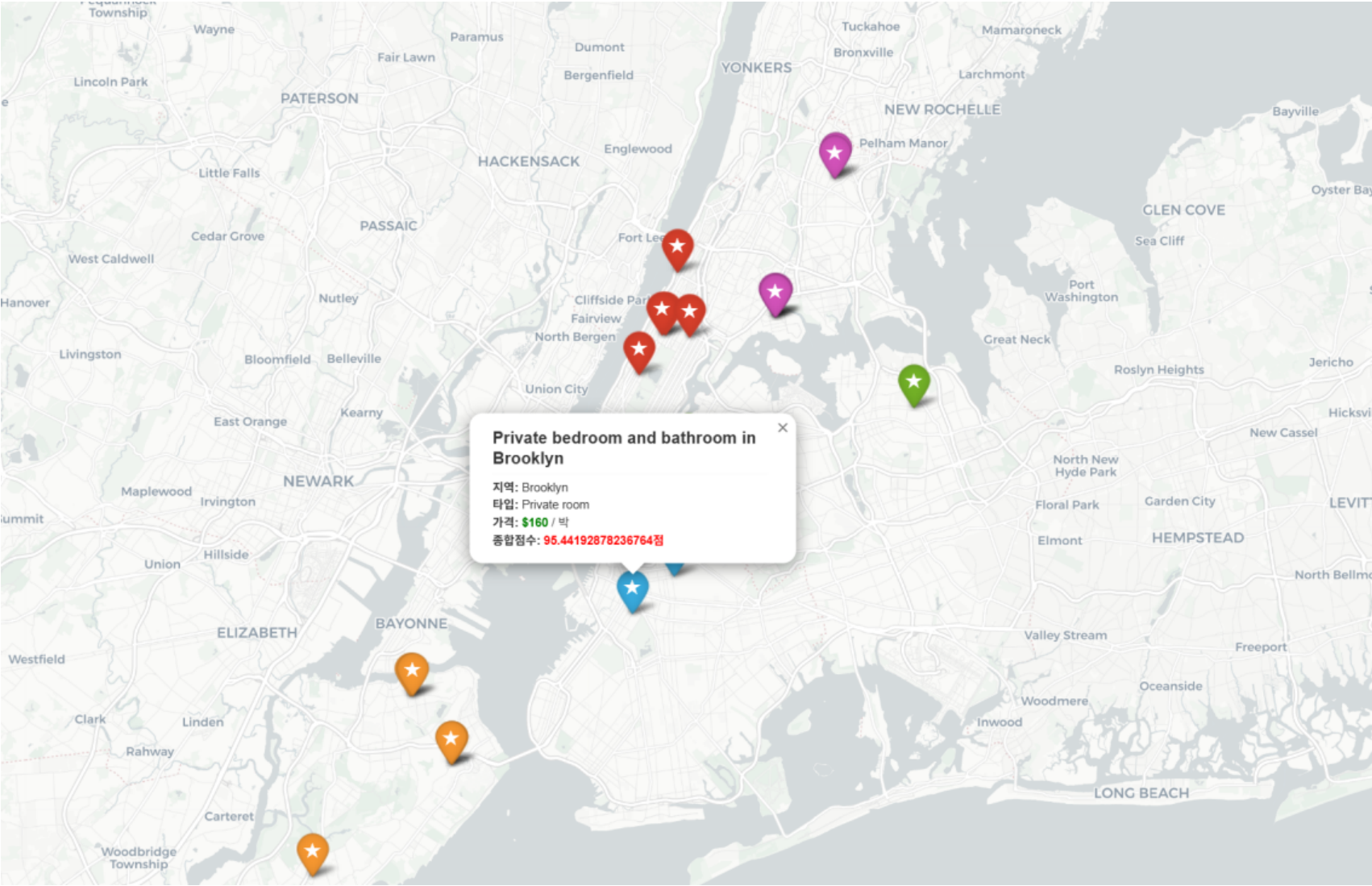
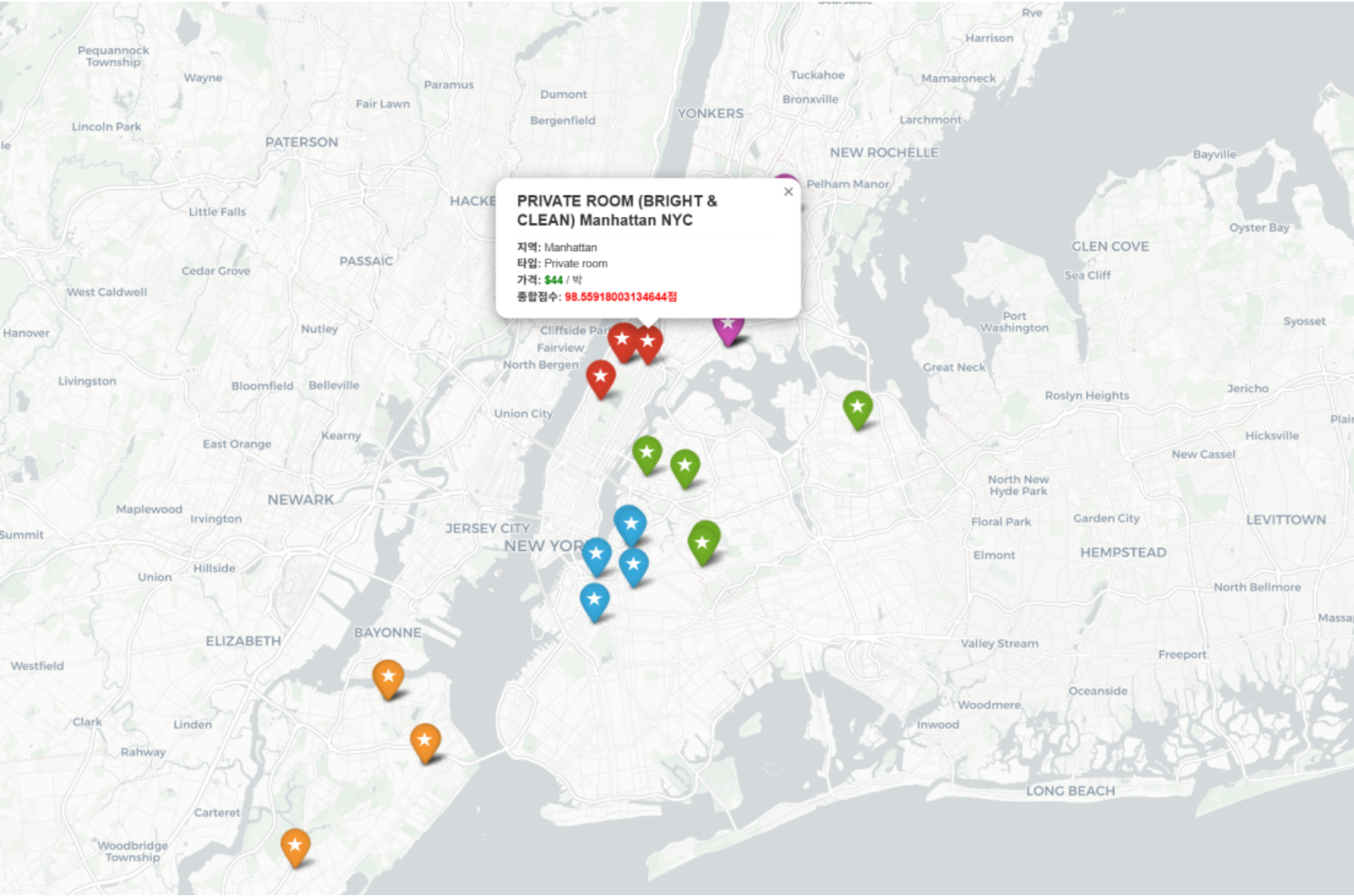
# 5. 완성된 지도 출력
display(nyc_map)
```

뉴욕 전체의 인터랙티브 지도를 생성

각 자치구별로 마커 색상을 다르게 지정하여
지역적 분포를 한눈에 알 수 있게 함

마커를 클릭했을 때 숙소의 이름, 가격, AI가
매긴 종합 점수가 나타나는 맞춤형 팝업 UI를
HTML과 CSS를 활용해 디자인

3. 뉴욕 지역별 TOP 5 숙소 시각화



고객 맞춤형 뉴욕 추천 숙소 TOP 10

	종합	가성비 중시	마케팅 우수
1위	Modern designed room in Manhattan! near B/C tr...	Modern designed room in Manhattan! near B/C tr...	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC
2위	Private Room in Williamsburg!	1 Bed Manhattan Apt. Minimum 7 DAYS	Modern designed room in Manhattan! near B/C tr...
3위	Private Room in Williamsburg Duplex	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	Private Room in Williamsburg!
4위	1 Bed Manhattan Apt. Minimum 7 DAYS	Location Top Manhattan	Private Room in Williamsburg Duplex
5위	Location Top Manhattan	Private room with a great location in Manhattan	SPACIOUS SUNNY ROOM NEAR SUBWAY AND HUDSON rIVER
⋮	⋮	⋮	⋮

뉴욕 지역별 추천 숙소 TOP 5

	Bronx	Brooklyn	Manhattan	Queens	Staten Island
1위	Cozy Private Room	Private Room in Williamsburg!	Modern designed room in Manhattan! near B/C tr...	Clean, Comfy Bright Small Bedroom	Share quiet, clean and spacious apt
2위	Blissfully Quiet Room	Private Room in Williamsburg Duplex	1 Bed Manhattan Apt. Minimum 7 DAYS	Bright-Spacious Private Room /30 min NYC	Room Available-Close to Manhattan-B
3위	Prolonged Traveler's Dream (a month first & more)	Private bedroom and bathroom in Brooklyn	Location Top Manhattan	Great Bedroom in Ridgewood, Queens!	Room Available-Close to Manhattan-A
4위	Prolonged Traveler's Dream (a month first & more)	Brooklyn Private Room 20 min to Manhattan	PRIVATE ROOM (BRIGHT & CLEAN) Manhattan NYC	One stop subway from mamhattan	room b
5위	Prolonged Traveler's Dream (a month first & more)	Luxury Private Bedroom in Brooklyn 2R-1	Private room with a great location in Manhattan	Renovated Private Room in Ridgewood Coliving S...	1-Bedroom Newly Renovated Cozy Apartment

기대 및 한계점

1

선정한 주제에 활용할만한 적절한 데이터가 많지 않아 결과적으로 R2값이 낮게 측정되었습니다. 숙소의 사진과 같은 비정형 데이터가 조금 더 주어졌다면 조금 더 유의미한 결과를 얻을 수 있었을 것으로 판단됩니다.

2

파생변수 생성 과정에서 단어를 임의로 선정하여 사용하였습니다. 실제 데이터에서 자주 등장하는 단어를 기반으로 변수를 구성했다면 모델의 설명력과 성능이 보다 향상되었을 것이라 판단됩니다.

3

이번 프로젝트에서 생성한 함수 모델은 주어진 데이터셋에서는 기대한 수준의 성능을 완전히 보여주지는 못했습니다. 그러나 설계 과정에서 확장성을 고려했기 때문에 다른 데이터셋이나 조건에서 활용 가능성을 보일 것이라고 기대합니다.

질의응답

Q&A